Open in app ↗

Medium

Level Up Coding · Follow publication

How to Build the Knowledge System Andrej Karpathy Uses (And What It's Actually For)

Here's what it actually does, how to build it yourself

5 min read · Apr 10, 2026

Tejas Sharma

Follow

Listen Share ••• More

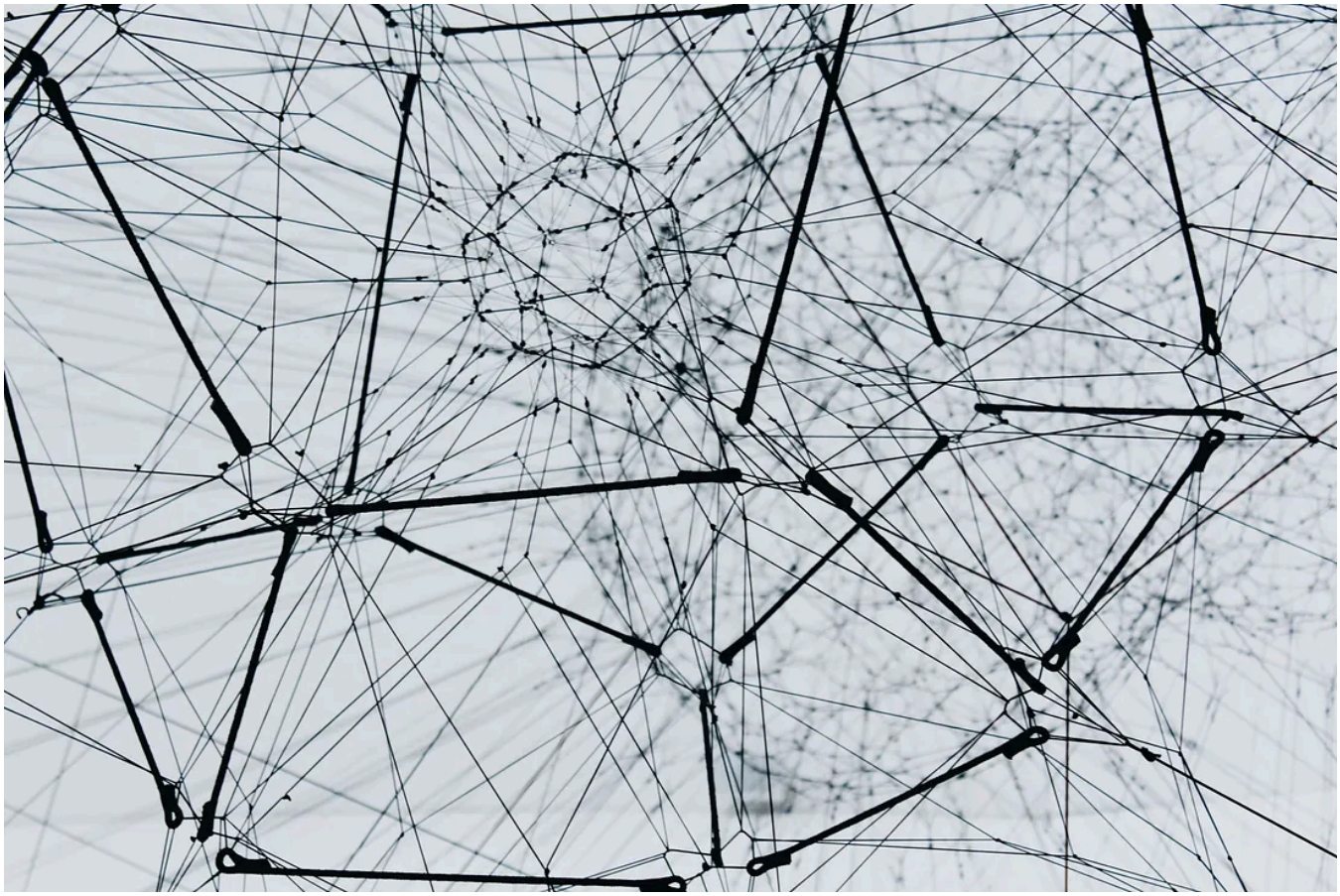

Photo by Alina Grubnyak on Unsplash

Andrej Karpathy posted a thread last week about how he manages knowledge. Millions of people read it. Most of them walked away thinking about the tools, the scripts, the markdown files, the Obsidian setup.

They missed what he was actually saying.

What he described wasn't a workflow, but a different relationship with knowledge entirely. One where you stop being the person who organizes everything and start being the person who asks questions. The AI does the rest. It reads your sources, builds the connections, writes the articles, maintains the whole thing.

You barely touch it.

Most people have never had that. They have folders. They have apps. Highlight libraries they never reread and note vaults that grow in one direction.

That's the gap Karpathy closed for himself.

And the fact that he had to build it with a hacky collection of scripts, as he put it himself, says everything about where we are right now.

What Most People Do With Knowledge

Most knowledge work looks the same. You read something useful. You save it. Then you move on.

The problem isn't that you're not saving enough. The problem is that nothing talks to anything else. The paper you read in January has no idea the article you saved in March said something that contradicts it. The insight you captured six months ago sits in a folder, waiting, while you rediscover the same idea from scratch in a different source three weeks later.

Knowledge without connection is just storage. And storage doesn't make you think better. It just makes you feel like you're on top of things.

What Karpathy Built and Why It's Different

What Karpathy built is different in one specific way: he's not the one maintaining it.

He dumps raw material into a directory. Articles, papers, repositories, images, whatever he's reading and researching. Then an LLM reads all of it and compiles a wiki.

It becomes a living document structure where concepts get their own articles, sources get linked, and connections get mapped. The AI automatically finds the relationships, writes the entries and keeps the whole thing coherent.

But uses Obsidian to view it instead of building it.

And here's why that distinction matters more than it sounds.

Every PKM system ever built assumes you are the one doing the connecting. Karpathy's system assumes the opposite.

And then when the wiki is big enough, you just ask it things. And it comes back with an answer. Your own knowledge now becomes quarriable.

How to Build It

Here's how to actually build it.

1. The raw directory

Create a folder. Call it raw/ or whatever you want. Everything goes in here. Articles you've clipped, papers you've downloaded, repositories you've bookmarked, images that matter.

The point is to get everything in one place the AI can read from. Karpathy uses the Obsidian Web Clipper extension to convert web articles into markdown files, which makes them easy for an LLM to process.

2. The compile step

This is the part that makes the whole thing work. You point an LLM at your raw directory and tell it to build a wiki. It reads everything, identifies the concepts, writes articles for them, links related ideas, adds backlinks, and organizes it all into a directory structure of markdown files.

You don't write any of this. The AI does. Your job is to feed it good sources.

3. The IDE layer

Karpathy uses Obsidian as the frontend to view everything. The raw material, the compiled wiki, the visualizations. The key thing to understand is that Obsidian here

is a reader, not a builder. You're not creating notes inside it. You're using it to navigate what the AI built.

4. Q&A against the wiki

Once the wiki is large enough, you can ask it complex questions. The LLM reads through the relevant articles, finds the connections, and gives you an answer drawn entirely from your own accumulated knowledge. No hallucinations from the open web. Just what you've already read, synthesized on demand.

5. Filing outputs back in

This is where the compounding starts. When the AI gives you an answer or generates a visualization or writes a summary, you file it back into the wiki. Every query enriches the base. Every answer becomes part of what the system knows. Over time the wiki doesn't just reflect what you've read. It reflects what you've thought.

6. Linting

Periodically you run what Karpathy calls health checks. The LLM scans the wiki for inconsistencies, fills gaps using web search, finds interesting connections that could become new articles. It's maintenance, but the AI does it. You just review what it surfaces.

What It Actually Does for Your Thinking

The obvious answer is retrieval. You can find things faster. But that's the least interesting part. Anyone who's used a decent search tool knows retrieval alone doesn't make you smarter.

What this system does that nothing else does is show you what you don't know yet.

When an AI reads across everything you've accumulated and starts mapping connections, it finds the gaps. The place where two ideas almost touch but don't. The question your research is circling but never lands on. Or the contradiction sitting quietly between two sources you saved six months apart.

That's the professional value. It help you think things you wouldn't have thought alone.

A lawyer with ten years of case notes doesn't have an information problem. They have a synthesis problem. A consultant who's advised thirty companies in the same industry isn't short on knowledge. They're short on a way to reason across all of it at once. A researcher who's read five hundred papers isn't missing data. They're missing the pattern that connects it.

That's what this system is actually for.

But There's The Catch

The thing is, building this is not simple.

You need to know how to work with a terminal. You need to understand how to prompt an LLM to compile and maintain a wiki incrementally. You need to set up directory structures, manage markdown files, and wire together tools that were never designed to talk to each other.

Karpathy himself called it a hacky collection of scripts.

And let's be honest, most people won't build this. Because the overhead of building the system is exactly the kind of thing that kills the system before it ever becomes useful.

And that's before you've added a single source.

That gap, between what this system does and what it takes to build it, is exactly what I had in mind when I built Constella. Same idea. Your sources go in, the AI builds the connections, you ask questions against everything you've accumulated. No terminal. No scripts. No configuration. Just the thinking part.

If Karpathy's system is what you want, Constella is probably the closest you can get without spending a weekend writing code.

Artificial Intelligence | Knowledge Management | Pkm | Productivity | Research

Follow

Published in Level Up Coding

322K followers · Last published 1 day ago

Coding tutorials and news. The developer homepage gitconnected.com && skilled.dev && levelup.dev

Follow

Written by Tejas Sharma

381 followers · 276 following

Helping people navigate information overload with constella.app.

Responses (4)

Andre Banki

What are your thoughts?

See all responses